UNIT 4

Sequential Modelling & Natural Language Representations

- Sequential Data Modelling — Sequence Representation and Processing
- Recurrent Neural Networks — RNN Architecture, BPTT, Truncated BPTT
- Gated Architectures — LSTM Networks and Gate Mechanisms
- Advanced Sequential Models — GRU Networks
- NLP Representations — Word Embeddings, Word2Vec (Skip-gram, CBOW), GloVe

Sequential Data Modelling

What Makes Data Sequential?

Sequential data is data in which the order of elements carries information.

Text is a sequence of words or characters, speech is a sequence of audio frames, a stock price is a sequence of daily values, DNA is a sequence of nucleotides, and a video is a sequence of images.

'Dog bites man' and 'Man bites dog' contain exactly the same words but mean different things, so any model for such data must be sensitive to order.

Sequences bring three challenges that the feedforward networks of ANN cannot handle well:

- Variable length — sentences have 3 words or 300; an MLP needs a fixed-size input vector.
- Order and context/Semantic Meaning — the meaning of a word depends on the words before (and sometimes after) it, so the model needs some form of memory.
- Position-independent patterns — a pattern such as 'not good' means the same thing whether it appears at position 2 or position 200, so the model should reuse the same parameters at every time step (parameter sharing), just as a CNN reuses a filter across space.

Sequence Representation

Neural networks consume numbers, so a raw sequence is converted in stages:

What happens	Example
Split the raw sequence into units: words, characters, or subwords	'the cat sat' -> ['the', 'cat', 'sat']
Assign every distinct token an integer id; reserve ids for special tokens	<pad>=0, <unk>=1, the=2, cat=3, sat=4 <pad> is used for making all sentence same and <unk> is used for unknown data
Replace each token by its id (or a one-hot vector)	[2, 3, 4]
Map each id to a dense learned vector (Embeddings)	3 -> [0.12, -0.8, 0.33, ...]

Sequence Processing

Because sequences differ in length, they are processed in batches using three tools: padding (append <pad> tokens so all sequences in a batch share one length), masking (a 0/1 matrix that tells the model and the loss function which positions are real tokens, so padding never contributes to the result)

The shape of the task also varies. The same recurrent machinery is arranged differently depending on how many inputs and outputs there are:

Pattern	Input -> Output	Example task
One-to-many	1 input -> sequence	Image captioning
Many-to-one	Sequence -> 1 output	Sentiment classification, spam detection
Many-to-many (aligned)	Sequence -> sequence, same length	Part-of-speech tagging, next-character prediction
Many-to-many (encoder-decoder)	Sequence -> sequence, different length	Machine translation, summarization

Practical Implementation — Tokenization, Padding, Masking, One-Hot Encoding

```
import numpy as np

sentences = ["the cat sat on the mat",
             "the dog barked",
             "a cat and a dog played in the garden"]

# 1. Tokenize and build a vocabulary (index 0 reserved for padding)
tokens = [s.split() for s in sentences]
vocab = {"<pad>": 0, "<unk>": 1}
for sent in tokens:
    for w in sent:
        vocab.setdefault(w, len(vocab))

# 2. Convert each sentence to a sequence of integer ids
ids = [[vocab.get(w, vocab["<unk>"]) for w in sent] for sent in tokens]

# 3. Pad to equal length so sequences can be batched, and build a mask
max_len = max(len(s) for s in ids)
padded = np.array([s + [0] * (max_len - len(s)) for s in ids])
mask = (padded != 0).astype(float)          # 1 = real token, 0 = padding
lengths = mask.sum(axis=1).astype(int)

print("Vocabulary size:", len(vocab))
print("Padded batch shape (batch, time):", padded.shape)
print(padded)
print("Sequence lengths:", lengths)

# 4. One-hot representation: (batch, time, vocab)
one_hot = np.eye(len(vocab))[padded]
print("One-hot tensor shape:", one_hot.shape)

# 5. Next-token prediction pairs: input = sequence[:-1], target = sequence[1:]
seq = ids[0]
inputs, targets = seq[:-1], seq[1:]
print("Inputs :", inputs)
print("Targets:", targets)
```

Recurrent Neural Networks

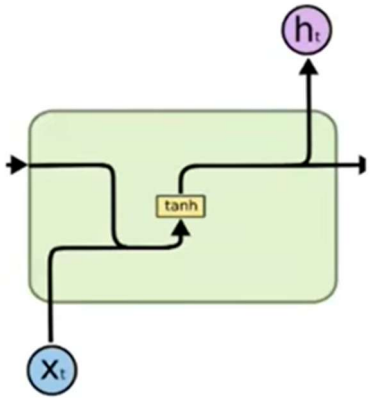
RNN Architecture

A Recurrent Neural Network processes a sequence one element at a time while maintaining a hidden state

h_t — a vector that acts as the network's memory of everything seen so far.

At each time step the new hidden state is computed from the current input x_t and the previous hidden state h_{t-1}, and the output is read off the hidden state.

Crucially, the same weight matrices are used at every time step, which is what lets the network handle any sequence length with a fixed number of parameters.



$$h_t = tanh(W_xh \cdot x_t + W_hh \cdot h_{t-1} + b_h)$$
$$\hat{y}_t = softmax(W_hy \cdot h_t + b_y)$$

Symbol	Meaning	Shape
x_t	Input at time t (one-hot vector or embedding)	(input_size)
h_t	Hidden state — the network's memory; $h_0 = 0$	(hidden_size)
W_xh	Input-to-hidden weights (shared across time)	(hidden_size, input_size)
W_hh	Hidden-to-hidden (recurrent) weights (shared across time)	(hidden_size, hidden_size)
W_hy	Hidden-to-output weights	(output_size, hidden_size)

Backpropagation Through Time (BPTT)

BPTT is simply backpropagation applied to the RNNs network.

The total loss is the sum of the per-step losses, and because the weights are shared, each weight's gradient is the sum of its contributions from every time step.

The subtlety is that the loss at time t depends on W_hh not only through h_t but also through every earlier hidden state h_k ($k \le t$), so the gradient must travel backward through all of those steps:

$$\partial L/\partial W_hh = \Sigma_t \Sigma_{k=1}{}^{t} (\partial L_t/\partial h_t) \cdot (\prod_{i=k+1}{}^{t} \partial h_i/\partial h_{i-1}) \cdot (\partial h_k/\partial W_hh)$$
$$\partial h_j/\partial h_{j-1} = diag(1 - h_j{}^2) \cdot W_hh$$

Condition	Effect on the product	Consequence
Largest singular value / spectral radius of W_hh < 1	Product shrinks exponentially with distance	Vanishing gradient: the network cannot learn long-range dependencies
Spectral radius of W_hh > 1	Product grows exponentially with distance	Exploding gradient: unstable training, NaN losses

Truncated BPTT

Full BPTT over a sequence of length T costs O(T) time and memory per update (all intermediate states must be stored), and the long gradient chains are numerically unstable.

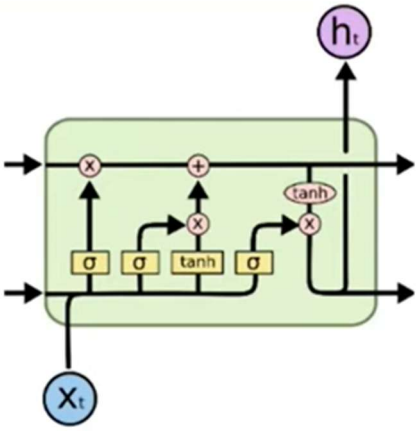
Truncated BPTT fixes both problems by cutting the sequence into chunks of length k (commonly 20–200). Two things happen at each chunk boundary:

- Forward pass: the hidden state is carried over, so the network still 'remembers' what came before and can use long-range context at inference time.
- Backward pass: the gradient is stopped at the boundary — it flows back at most k steps — so memory is bounded and each update is cheap.

The trade-off is that the network cannot learn dependencies longer than k steps through gradient signal alone, although it can still exploit longer context through the carried-over state.

Gated Architectures: LSTM Networks

Long Short-Term Memory networks (Hochreiter & Schmidhuber, 1997) were designed specifically to solve the vanishing-gradient problem of the plain RNN. The core idea is to give the network a second, protected memory channel — the cell state c_t — that information can flow along almost unchanged, and to control what enters, leaves and is erased from that channel using learned gates.



Gate Mechanisms

A gate is a vector of numbers between 0 and 1, produced by a sigmoid, that is multiplied element-wise with another vector. A value near 1 means 'let this through' and a value near 0 means 'block this', so each gate acts as a learned, soft, differentiable switch. An LSTM has three gates plus a candidate memory, all computed from the previous hidden state and the current input:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad \textit{(forget gate)}$$
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad \textit{(input gate)}$$
$$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad \textit{(candidate memory)}$$
$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad \textit{(output gate)}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad \textit{(cell state update)}$$
$$h_t = o_t \odot \tanh(c_t) \quad \textit{(hidden state / output)}$$

Component	Question it answers	Behaviour
Forget gate f_t	What should I erase from long-term memory?	$f_t \approx 0$ clears a memory cell; $f_t \approx 1$ keeps it untouched
Input gate i_t	How much of the new information should I store?	Scales the candidate $\tilde{c}_t$ before it is added to the cell state
Candidate $\tilde{c}_t$	What new content could be stored?	A tanh-squashed proposal computed from the current input and previous state
Output gate o_t	How much of the memory should be exposed right now?	Controls what part of c_t becomes the visible hidden state h_t

Component	Question it answers	Behaviour
Cell state c_t	The long-term memory	Updated additively, so information and gradients can flow across many steps

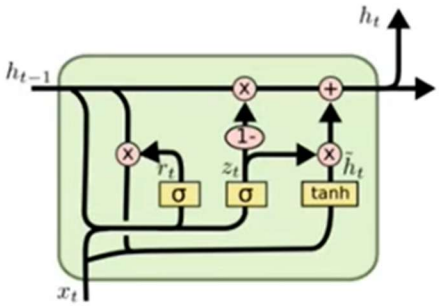
Extensions

- Stacked (deep) LSTMs: the hidden-state sequence of one LSTM layer is fed as the input sequence of the next, building hierarchical representations.
- Bidirectional LSTMs: two LSTMs read the sequence left-to-right and right-to-left and their states are concatenated, giving each position both past and future context (used for tagging and classification, not for left-to-right generation).

Advanced Sequential Models: GRU Networks

The Gated Recurrent Unit (Cho et al., 2014) keeps the central idea of the LSTM — additive, gated updates that let gradients flow — but simplifies the design. It removes the separate cell state (the hidden state itself is the memory) and merges the forget and input gates into a single update gate, leaving only two gates in total.

GRU Architecture



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z) \quad \textit{(update gate)}$$
$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r) \quad \textit{(reset gate)}$$
$$\tilde{h_t} = \tanh(W_h \cdot [r_t \odot h_{t-1}, x_t] + b_h) \quad \textit{(candidate state)}$$
$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h_t} \quad \textit{(new hidden state)}$$

- Update gate z_t decides how much of the old state to keep versus how much to replace with the new candidate. The final line is a learned interpolation: $z_t \approx 0$ copies the old state forward unchanged (long-term memory, with gradients flowing freely), $z_t \approx 1$ overwrites it with new content. This single gate does the job of both the LSTM's forget and input gates.
- Reset gate r_t controls how much of the previous state is used when computing the candidate. $r_t \approx 0$ makes the unit ignore its history and behave as if reading the start of a new sequence — useful for dropping irrelevant context.

LSTM vs GRU

Aspect	LSTM	GRU
Gates	3 (forget, input, output)	2 (update, reset)
Separate memory cell	Yes (c_t and h_t)	No (h_t only)
Weight blocks per layer	4	3 (about 25% fewer parameters)
Training speed	Slower	Faster
Strengths	More expressive; strong when very long dependencies matter	Fewer parameters; often matches LSTM accuracy, especially on smaller datasets
Rule of thumb	Try first on long or complex sequences	Try first when data or compute is limited; compare both empirically

NLP Representations

Word Embeddings

The problem with sparse representations

A one-hot vector has the length of the vocabulary (often 50,000+), contains a single 1, and is orthogonal to every other word's vector. This causes two problems: the vectors are huge and wasteful, and — more importantly — they contain no notion of similarity. 'King' is exactly as far from 'queen' as it is from 'banana'. Count-based alternatives such as Bag-of-Words and TF-IDF are also sparse and ignore word order and meaning.

The dense embedding solution

A word embedding maps each word to a short dense vector (typically 50–300 dimensions) in which geometric closeness reflects similarity of meaning. Words that appear in similar contexts (cat/dog, apple/banana) end up with similar vectors, so embeddings can be learned automatically from unlabelled text simply by predicting words from their neighbours.

Property	One-hot	Dense embedding
Dimension	Vocabulary size (10^4–10^6)	50–300
Values	Mostly zeros (sparse)	Real numbers (dense)
Captures similarity?	No — all words equally distant	Yes — similar words are close (cosine similarity)
Learned from data?	No	Yes
Generalization	None between words	Knowledge transfers between related words

Word2Vec

Word2Vec learns embeddings with a shallow, fast neural network trained on a simple prediction task over a sliding context window of size m. Each word has two vectors: an input vector v_w (used when the word is the input) and an output vector u_w (used when it is the target). After training, the input vectors (sometimes averaged with the output vectors) are kept as the embeddings. There are two architectures, which are mirror images of one another.

CBOW (Continuous Bag-of-Words)

CBOW predicts the centre word from its surrounding context words. The context vectors are averaged — order within the window is ignored, hence 'bag of words' — and the average is used to score every word in the vocabulary through a softmax. CBOW trains quickly and gives good representations for frequent words because averaging smooths out noise.

Skip-gram

Skip-gram reverses the task: given the centre word, it predicts each of the words in its context window. Every (centre, context) pair is a separate training example, so the model sees many more updates per occurrence of a rare word, which is why Skip-gram tends to represent rare words and small datasets better than CBOW, at a higher training cost.

Aspect	CBOW	Skip-gram
Task	Context words -> centre word	Centre word -> each context word
Training examples per window	1	Up to 2m
Speed	Faster	Slower
Best for	Large corpora, frequent words	Smaller corpora, rare words

GloVe (Global Vectors)

Word2Vec learns from local context windows, one window at a time.

GloVe instead first collects global co-occurrence statistics from the entire corpus into a matrix X, where X_{ij} is how often word j appears in the context of word i, and then fits word vectors directly to those statistics. It is essentially a weighted matrix-factorization method on log co-occurrence counts.

The key insight is that meaning is captured not by raw co-occurrence probabilities but by their ratios. For the words ice and steam, a probe word such as 'solid' is much more likely to appear near ice, 'gas' near steam, while a word like 'water' appears near both and a word like 'fashion' near neither. The ratio P(k|ice) / P(k|steam) is large for solid-like words, small for gas-like words and close to 1 for words related to both or neither, so it separates the relevant dimensions of meaning from the noise. GloVe builds vectors so that their dot products reproduce the log of the co-occurrence counts, which makes vector differences encode these ratios.

Aspect	Word2Vec	GloVe
Learns from	Local context windows, streamed one at a time	Global co-occurrence matrix built from the whole corpus
Objective	Predictive: classify the true word / pair (softmax or negative sampling)	Count-based: weighted least squares on log co-occurrence
Training signal	Each window occurrence	Each distinct word pair, weighted by f(X)
Memory	Low — no matrix needed	Needs the co-occurrence matrix (sparse)
Quality	Both give comparable embeddings on most tasks	Both give comparable embeddings on most tasks
Limitation shared by both	One static vector per word — no handling of polysemy or context	Same